ENGENHARIA DE SOFTWARE · UNICV – CENTRO UNIVERSITÁRIO CIDADE VERDE

Em Dia

Radar de vencimentos para adultos com TDAH

Entrega 3 – Documentação Técnica da Solução e Resultados dos Testes

Trabalho Multidisciplinar · UX – Experiência do Usuário + Tópicos Especiais

INTEGRANTES

Felipe Moreira dos Santos · RA 443777-2026

Felipe Nadab Goldberg Barbosa · RA 156888-2023

Maringá/PR · Outubro de 2026

Sumário

Seção	Conteúdo
1.	Introdução e visão geral da solução
2.	Arquitetura implementada e tecnologias
3.	Banco de dados
4.	API (E01 a E25)
5.	Regras de negócio (RN01 a RN15)
6.	Rotina diária de avisos
7.	Frontend web e app mobile
8.	Autenticação, segurança e LGPD
9.	Resultados dos testes
10.	Desvios e bugs
11.	Publicação online
12.	Conclusão, dificuldades e aprendizados

Sobre este documento

Esta é a terceira entrega do trabalho multidisciplinar. Ela descreve a solução construída a partir da especificação técnica da Entrega 2 — arquitetura, banco, API, regras, telas e segurança — e registra os resultados dos testes, os desvios e os bugs encontrados.

Integrante	RA	Responsabilidade principal
Felipe Moreira dos Santos	443777-2026	API .NET, banco de dados, autenticação, rotina de avisos, testes da API
Felipe Nadab Goldberg Barbosa	156888-2023	Frontend React, app mobile, metáforas e acessibilidade, testes do front e de usabilidade

1. Introdução e visão geral da solução

O **Em Dia** reúne os prazos da vida prática de adultos com TDAH — documentos, veículo, casa, saúde, finanças e garantias — e avisa antes do vencimento, com tempo visível, passos pequenos e avisos que insistem. Esta entrega implementa a solução especificada na **Entrega 2**, seguindo o contrato da API, o modelo de dados, as regras de negócio, a segurança e as telas daquele documento. As diferenças estão registradas na seção 10.

Radar em 4 anéis (RN03)

Centro = vencido; anel 1 = até 30 dias; anel 2 = 31 a 90; anel externo = mais de 90. Seis fatias, uma por área. Cada ponto tem forma, cor e texto próprios.

Modo foco (RN04)

Só os 3 itens mais urgentes: vencidos primeiro (os mais atrasados antes), depois os de menor prazo.

Check-up

17 perguntas sim/não organizadas por área (Veículo, Documentos, Casa, Saúde, Finanças e Garantias), uma por tela, com a opção de pular a área; depois datas, km e veículo; por fim o resultado por área ("4 de 6 áreas em dia"). Área sem itens aparece como "Sem itens" e não conta como em dia. Não duplica itens ao ser refeito (RN10).

Quatro tipos de vencimento (RN01)

Data única, recorrente por tempo, uso ou tempo (km ou dias, o que vier primeiro) e garantia (compra + fabricante + 90 dias).

Concluir e renovar (RN05–RN07)

Histórico, novo prazo, passos desmarcados e mensagem positiva; passos pendentes pedem confirmação; a km do veículo nunca diminui.

Avisos e LGPD

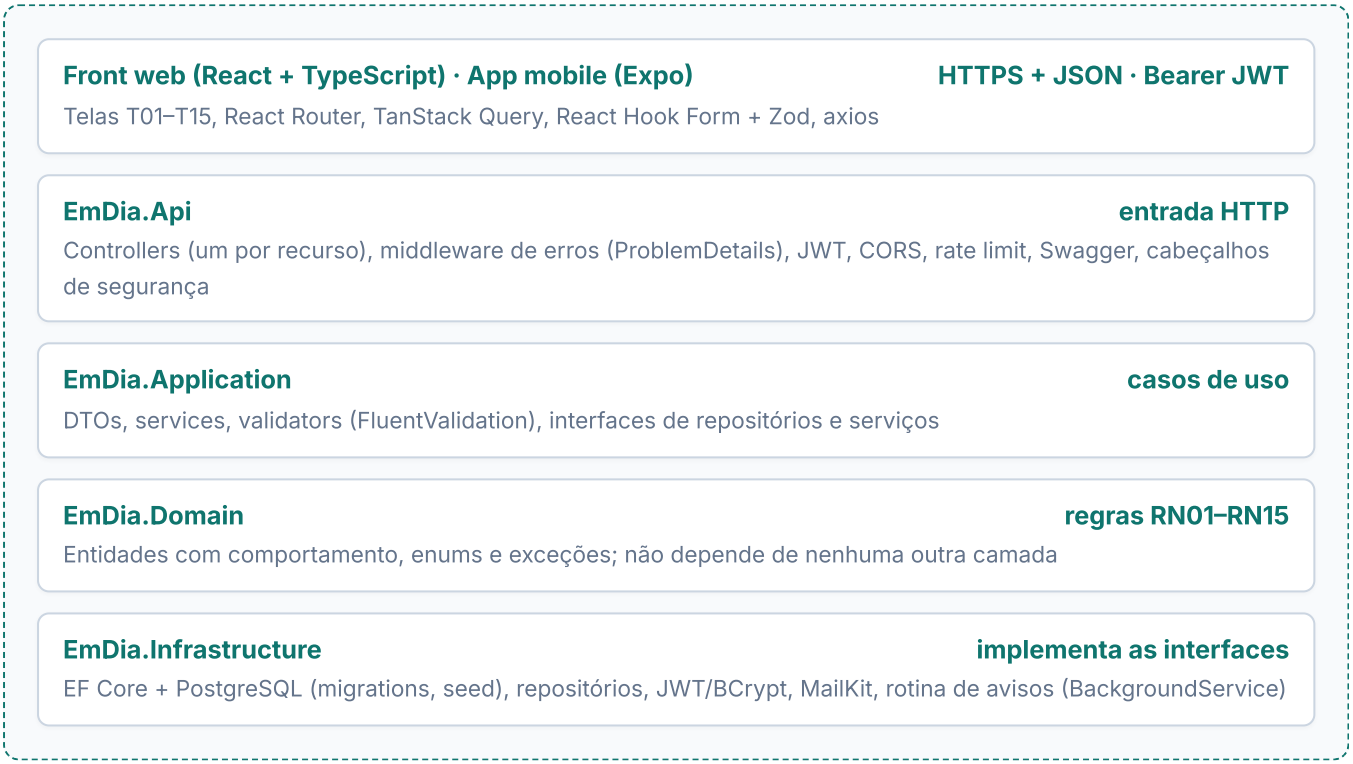
Rotina diária às 8h com aviso antecipado e reavisos limitados (RN08/RN09). Exclusão da conta e de todos os dados com confirmação de senha (RF18).

Onde está cada parte

API e testes em `api/`; front web em `web/`; app mobile (Expo) em `mobile/`; desvios, bugs e resultado de cada teste em `docs/`. Documentação interativa da API em `/swagger`.

2. Arquitetura implementada e tecnologias

Aplicação em duas partes, como na Entrega 2: **frontend React (SPA)** e **API ASP.NET Core** em camadas baseadas em Clean Architecture. Um **app mobile em React Native/Expo** (extra) usa a mesma API. **Regra de dependência:** Api → Application → Domain; Infrastructure → Application e Domain — por isso as regras de vencimento são testadas sem banco e sem API.



Parte	Tecnologia
API	.NET 10 (LTS) · ASP.NET Core Web API
Dados	Entity Framework Core 10 + Npgsql 10 · PostgreSQL 17
Validação	FluentValidation (API) · Zod (front e app)
Segurança	JWT Bearer (HS256) · BCrypt (custo 12) · Rate Limiter do ASP.NET Core
Documentação e e-mail	Swagger / OpenAPI em `/swagger` · MailKit (SMTP)
Frontend	React 19 · TypeScript · Vite · React Router · TanStack Query · React Hook Form · Zod · axios
App mobile	React Native · Expo SDK 58 (Expo Go) · React Navigation · expo-secure-store
Testes	xUnit · WebApplicationFactory · Testcontainers (PostgreSQL) · Vitest · Testing Library
Publicação e CI	Docker · GitHub Actions · Vercel (site) · Render (API) · Neon (PostgreSQL)

3. Banco de dados

PostgreSQL criado e versionado por migrations do EF Core (`CriacaoInicial`, `AjustesEntrega2` e `CheckupPorArea`). São **12 tabelas** em quatro grupos, com nomes em português no formato `snake_case`.

Grupo	Tabelas
Conta	`usuarios`, `preferencias_aviso`, `refresh_tokens`, `checkups`
Catálogo	`categorias` (6 áreas fixas), `modelos_item`, `passos_modelo`
Vencimentos	`itens`, `passos`, `historicos`, `avisos`
Veículos	`veiculos`

- **Chaves:** uuid geradas pela API (`categorias` usa smallint). Todas as chaves estrangeiras do DER estão no banco: excluir um usuário apaga em **cascata** todos os seus dados; categorias, modelos e veículos com itens ficam protegidos.
- **CHECKs:** km ≥ 0; média de 1 a 20.000; intervalos > 0; antecedência de 0 a 365; reavisos de 0 a 5; intervalo de reaviso de 1 a 7; valor gasto ≥ 0; campos obrigatórios por tipo de vencimento.
- **Unicidade:** e-mail; categoria; modelo por categoria; apelido do veículo por usuário; passo por (item, ordem); aviso por (item, vencimento, número); hash do refresh token.
- **Índices:** `itens (usuario_id, situacao, proximo_vencimento_data)` para o Radar e a rotina de avisos; `avisos (item_id, vencimento_referencia)`; índices em todas as chaves estrangeiras.
- **Status calculado:** Vencido, Atenção, Em dia e Encerrado **não são gravados**; são calculados na leitura, porque dependem da data de hoje e da km atual.
- **Dados iniciais:** 6 categorias e 19 modelos prontos com passos, ligados às 17 perguntas do check-up (troca de óleo, IPVA, CNH, passaporte, aluguel, filtro do purificador, dentista, vacina do pet, declaração do IR, IPTU, garantias e outros).

4. API (E01 a E25)

Base `/api/v1`, JSON em camelCase, enums como texto, datas ISO 8601 e erros no formato ProblemDetails (RFC 9457). Toda rota exige autenticação, exceto registrar, login e refresh. Recursos de outra pessoa respondem 404.

#	Método	Rota	Objetivo
E01	`POST`	`/auth/registrar`	Criar conta
E02	`POST`	`/auth/login`	Entrar (5 tentativas/min por IP)
E03	`POST`	`/auth/refresh`	Renovar o token de acesso
E04	`POST`	`/auth/logout`	Sair
E05	`GET`	`/usuarios/me`	Conta e preferências
E06	`PUT`	`/usuarios/me/preferencias`	Alterar preferências de aviso
E07	`DELETE`	`/usuarios/me`	Excluir conta e todos os dados (exige senha)
E08	`GET`	`/categorias`	Listar categorias
E09	`GET`	`/modelos`	Listar modelos prontos
E10	`GET`	`/checkup/perguntas`	Perguntas do check-up
E11	`POST`	`/checkup`	Fazer o check-up e criar os itens (uma transação)
E12	`GET`	`/radar?foco=`	Radar e modo foco
E13	`GET`	`/itens`	Listar itens (filtros: categoria, status)
E14	`GET`	`/itens/{id}`	Detalhe com passos e histórico recente
E15	`POST`	`/itens`	Criar item
E16	`PUT`	`/itens/{id}`	Editar item
E17	`DELETE`	`/itens/{id}`	Excluir item
E18	`POST`	`/itens/{id}/concluir`	Concluir e renovar
E19	`PATCH`	`/itens/{id}/passos/{passoId}`	Marcar ou desmarcar passo
E20	`GET`	`/veiculos`	Listar veículos
E21	`POST`	`/veiculos`	Cadastrar veículo
E22	`PUT`	`/veiculos/{id}`	Editar veículo
E23	`PATCH`	`/veiculos/{id}/km`	Atualizar a quilometragem
E24	`DELETE`	`/veiculos/{id}`	Excluir veículo (bloqueado se houver itens)
E25	`GET`	`/historico`	Listar conclusões (filtros: item, de, até)

Códigos: 200/201/204; 400 com `errors` por campo; 401; 404; 409 (e-mail ou apelido repetido, veículo com itens); 422 com `code` (ex.: `PASSOS_PENDENTES`); 429; 500 com mensagem genérica.

5. Regras de negócio (RN01 a RN15)

Todas ficam na camada Domain (`Item` , `Veiculo` , `RadarDomain` , `AvisoDomain` , `CheckupDomain`).

Código	Implementação
RN01	Grava o prazo por tempo e a km alvo; a data estimada pela km é calculada na leitura: (km que falta ÷ média) × 30 dias, valendo o que vier primeiro.
RN02	Vencido se a data passou ou a km foi atingida; Atenção dentro da antecedência; Em dia nos demais casos.
RN03	Centro, até 30 dias, 31 a 90 dias, mais de 90 dias.
RN04	Modo foco: 3 itens mais urgentes.
RN05	Concluir: histórico, nova base, novo prazo e passos desmarcados; data única encerra ou recebe nova validade; garantia encerra.
RN06	Passos pendentes sem confirmação → `PASSOS_PENDENTES` (422).
RN07	A km do veículo nunca diminui; concluir com km maior atualiza o veículo.
RN08 / RN09	Aviso antecipado e reavisos a cada intervalo, até o máximo por ciclo; o ciclo recomeça a cada novo vencimento.
RN10	O check-up não duplica item do mesmo modelo e veículo.
RN11	Resultado do check-up por status e por área; área sem itens fica "Sem itens" e não conta como em dia.
RN12	Repositórios filtram pelo dono; recurso alheio → 404.
RN13	Veículo com itens não pode ser excluído (409).
RN14	Passos do modelo copiados ao criar o item (editáveis).
RN15	Mensagens positivas, ex.: "Boa! Próxima vez: 95.200 km ou 15/10/2027, o que vier primeiro."

6. Rotina diária de avisos

- `AvisosWorker` (BackgroundService) dispara a rotina às **8h de America/Sao_Paulo**.
- Para cada usuário com e-mail ativado, envia o aviso antecipado (número 0) quando o item entra em Atenção ou Vencido e, sem conclusão, reavisos a cada `intervalo_reaviso_dias` , até `max_reavisos` por ciclo.
- Cada envio fica em `avisos` , único por (item, vencimento, número): rodar duas vezes no dia não duplica e-mails; uma falha de envio é refeita no dia seguinte.
- O e-mail é curto, positivo e traz o próximo passo pequeno. Sem SMTP configurado, as mensagens só vão para o log.

7. Frontend web e app mobile

7.1 Telas (T01–T15) e navegação

Entrar, criar conta, check-up (perguntas, completar dados, resultado), Radar, modo foco, itens, novo item, detalhe, editar, concluir (janela), histórico, perfil e veículos — nas rotas da spec §5.1. Menu inferior no celular (lateral no computador): **Radar · Itens · + Novo · Histórico · Perfil**. Rotas privadas levam ao login, e o **portão do check-up** leva ao check-up enquanto ele não for feito.

7.2 Integração com a API

Cliente axios com o token de acesso só em memória; em 401, uma tentativa de `/auth/refresh` e a requisição é repetida; se falhar, volta ao login. TanStack Query guarda Radar e itens e recarrega depois de cada ação. Marcar passo usa atualização otimista. Erros 400 vão para os campos; 422 `PASSOS_PENDENTES` vira a pergunta "Concluir mesmo assim?".

7.3 Acessibilidade (RNF02 / RNF03)

- Status sempre com **cor + símbolo + texto**; pontos do Radar com forma diferente por status e descrição para leitores de tela.
- Áreas de toque de 44 px; uma pergunta por tela no check-up, toda feita só com Tab e Enter.
- Mensagens de erro abaixo do campo, ligadas por `aria-describedby`, e foco no primeiro campo inválido.

7.4 App mobile (Expo Go)

O app em `mobile/` usa a mesma API e as mesmas validações. No **Expo Go**, descobre sozinho o IP do computador pela rede Wi-Fi (o mesmo do servidor Metro) e chama a API em `http://<IP>:5196`. Como o app nativo não tem cookies de navegador, o token de renovação vai no corpo da resposta e fica no **SecureStore** do aparelho (DV17).

> **Fluxos verificados (F1–F4)**
> Primeiro acesso (cadastro → check-up → resultado → Radar), concluir item com passo pendente, criar item a partir de modelo e atualizar a quilometragem — executados com a API e o front rodando juntos, também na largura de celular.

8. Autenticação, segurança e LGPD

Risco	Medida
Força bruta	5 tentativas de login por minuto por IP (429) e bloqueio da conta por 15 minutos após 5 falhas.
Senhas	Mínimo de 8 caracteres com letra e número; BCrypt custo 12.
Descoberta de contas	Login com mensagem genérica.
Dados de outra pessoa	Id do usuário só a partir do token; consultas filtradas; 404 para recursos alheios.
Sessão	JWT de 15 minutos (HS256, 30 s de tolerância); refresh de 7 dias com rotação; reuso revoga todos os tokens do usuário.
XSS / CSRF	Token de acesso fora do localStorage; refresh em cookie HttpOnly, Secure, SameSite=Strict, Path=/api/v1/auth; Content-Security-Policy.
Interceptação	HTTPS com HSTS fora do desenvolvimento.
CORS	Só a origem do frontend configurada.
Entrada maliciosa ou enorme	Validação de tamanho e formato em todos os campos; corpo limitado a 64 KB.
Vazamento por erro ou log	ProblemDetails sem stack trace; logs sem senha, token ou hash.
Segredos	Chave do JWT e senha do SMTP fora do código (user-secrets e variáveis de ambiente).
LGPD	Só os dados necessários; nada de saúde ou TDAH é guardado; exclusão da conta e de todos os dados (E07).

9. Resultados dos testes

"Hoje" é fixado em **15/10/2026** pela interface `IRelogio` nos testes de unidade e de integração; os testes de integração usam um PostgreSQL descartável.

Nível	Ferramenta	Testes	Resultado
Unitário (Domain)	xUnit	50 (U01–U15 + 35 complementares)	50/50
Integração da API	xUnit + WebApplicationFactory + PostgreSQL	52 (CT01–CT46 + extras)	52/52
Frontend web	Vitest + Testing Library	17 (CT47–CT51 + validações)	17/17
App mobile	Vitest	11 (validações e datas)	11/11
Total		**130**	130/130

Critérios de aceite (§6.6)

U01–U15 e casos de prioridade alta aprovados; cobertura do `EmDia.Domain` de **97,3%** das linhas (mínimo 80%); fluxos F1–F4 sem erro; CT08 e CT44 aprovados.

Destaques

U02 (km vem primeiro → Atenção em 21/10/2026), U13 (avisos em D, D+3 e D+6), CT11 (reuso de refresh revoga tudo), CT42 (exclusão da conta apaga tudo), CT51 (check-up só com teclado).

O resultado de cada caso (U01–U15, CT01–CT51) está em `docs/testes.md`. O CI do GitHub roda a cada push os testes da API, a imagem Docker da API com PostgreSQL, o front e o app (tipos, testes e `expo-doctor`).

10. Desvios e bugs

10.1 Principais desvios (lista completa em docs/desvios.md)

Código	Desvio e motivo
DV03	Campo opcional `novaDataVencimento` no E18, para renovar itens de data única (RN05).
DV09	A pergunta "Você tem carro ou moto?" (`TEM_CARRO`) cobre carro e moto, porque `codigo_checkup` é uma única coluna.
DV10	Check-up por área, com 17 perguntas e modelos complementares aos exemplos da §2.7.
DV11	Respostas com alguns campos a mais que os exemplos da §3.4 (nenhum removido).
DV18	Área sem itens não conta como "em dia"; pular o check-up todo mostra o Radar vazio.
DV12	Refazer o check-up reaproveita o veículo de mesmo apelido (CT17).
DV17	No app nativo, o refresh token vai no corpo e fica no SecureStore; o navegador continua só com cookie.
DV19	Publicação atrás de proxy (Vercel → Render): IP real no rate limit de login e migrations ao subir.

10.2 Bugs corrigidos (detalhes em docs/bugs.md)

- Data por km calculada com o relógio do servidor e gravada no banco (RN01).
- Exclusão da conta sem apagar os dados — faltavam chaves estrangeiras do DER.
- Concluir item respondia 500 (histórico tratado como registro existente).
- Erro 400 sem o campo `errors`; contador de falhas de login sem zerar após o bloqueio.
- Foco inicial do check-up, destaque do resultado na área errada e observação em branco no histórico.
- App mobile chamando rotas que não existem na API e com bibliotecas fora da versão do Expo Go.
- CI do GitHub vermelho: banco padrão do Testcontainers apagado pelo próprio teste (BUG-14).
- Publicação no Render: caminho do Dockerfile e connection string com aspas (BUG-15).

11. Publicação online

O sistema está publicado com HTTPS, em serviços gratuitos (passo a passo em `docs/deploy.md`):

Parte	Serviço	Endereço
Site (React)	Vercel	em-dia-projeto-facul.vercel.app
API (.NET em Docker)	Render	emdia-api-felipemsantoss.onrender.com/swagger
Banco (PostgreSQL 17)	Neon	AWS US East 1, a mesma região da API

- O Vercel repassa `/api/*` para o Render: o navegador vê um único endereço e o cookie de renovação (`SameSite=Strict`) continua valendo.
- A API aplica as migrations e os dados iniciais ao subir; a chave do JWT é gerada pelo Render e nunca entra no repositório.
- Atrás do proxy, a API lê o IP real do usuário: o limite de 5 tentativas de login continua por pessoa (DV19).
- No plano gratuito, a API "dorme" após 15 minutos sem uso e leva cerca de 1 minuto para acordar.

12. Conclusão, dificuldades e aprendizados

A solução implementa o que a Entrega 2 especificou — contrato da API, modelo de dados, regras de negócio, segurança e telas — com **130 testes automatizados aprovados**, cobertura de 97% nas regras de negócio e o sistema publicado online. As duas metáforas (Radar e Check-up) orientam as telas, e a acessibilidade foi verificada por testes (status com texto e símbolo, uso só com teclado).

Dificuldades encontradas

- Manter o código fiel à especificação: rotas, chaves estrangeiras e validações da primeira versão divergiam e foram reescritas.
- Datas: "hoje" no fuso de São Paulo e prazos por data ou por km; foi preciso um relógio controlável (`IRelogio`) nos testes.
- App no celular: versões exatas do Expo Go, sessão sem cookies e teclado do iPhone cobrindo os botões.
- Ambiente: testes que passavam no computador e falhavam no GitHub; erros de configuração na publicação.

Aprendizados

- Especificar antes de programar: o contrato da API guiou o front, o app e os testes.
- Regras no Domain, sem banco nem API, deixam os testes rápidos e confiáveis.
- Testes de integração com banco real acham erros que testes isolados não acham.
- Diagnosticar pelos logs e reproduzir o erro antes de corrigir.
- Segredos fora do repositório e acessibilidade desde a primeira tela.

Próximos passos: testes de usabilidade com usuários e métricas de UX; um servidor SMTP para os avisos por e-mail; uma versão instalável do app mobile (build com EAS); e uma hospedagem em que a API não durma.